

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Project Title

Public Transit Planner

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

Kartheek Reddy (2520090015)

Danush Raj (2520080046)

Team Number: 30

Section: 10

Under the guidance of

Mrs. Chandusha Kanda

Asst. Professor, CS&IT

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

DECLARATION

The Mini Project Report entitled “**Public Transit Planner**” is a record of Bonafide work of **Kartheek Reddy – 2520090015, Danush Raj – 2520080046**, submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Kartheek Reddy – 2520090015

Danush Raj – 2520080046

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the mini project based report entitled “**Public Transit Planner**” is a bonafide work done and submitted by **Kartheek Reddy – 2520090015, Danush Raj – 2520080046** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation goes to our guide, **Mrs. Chandusha Kanda**, Asst. Professor, Department of CS&IT, whose tremendous support, encouragement and guidance cannot be expressed in words.

We express our gratitude to **Dr. Khaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Karthek Reddy – 2520090015

Danush Raj – 2520080046

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	15
8.	Conclusion & Future Scope	20
9.	References	21

1. ABSTRACT

The Public Transit Planner is a data driven system designed to optimize public transportation routes and timetables while operating within fixed fleet and budget constraints. The project begins by collecting and cleaning transit stop, passenger demand, and travel time data to ensure accuracy and consistency. Using graph based routing and optimization techniques, the system identifies efficient routes that maximize service coverage and minimize passenger travel time and operating costs. It also generates practical timetables based on demand patterns and vehicle availability. The planner evaluates performance using key metrics such as route coverage, average waiting time, travel time, fleet utilization, and operational expenses. Interactive reports and visualizations help compare different planning scenarios and support informed decision making. This project demonstrates the application of data preprocessing, optimization algorithms, and data analytics in addressing real world transportation challenges. The proposed system improves accessibility, enhances service efficiency, and provides a cost effective decision-support solution for modern public transit planning.

Team Name: Team 30 (Section 10)

Team Logo (if any): —

Team Members:

- Kartheek Reddy, 2520090015
- Danush Raj, 2520080046

2. INTRODUCTION

Public transport is the backbone of urban mobility. A bus network that is poorly designed leaves neighbourhoods without service, forces passengers into long waits and detours, and wastes scarce buses on low-demand corridors. Planning agencies, however, must work within hard limits: a fixed fleet of vehicles and a fixed daily operating budget. The central question is therefore not “what is the best possible network?” but “what is the best network we can run with the buses and money we actually have?”

In the literature this is studied as the *Transit Network Design and Frequency Setting Problem* (TNDFSP). It combines four linked decisions: (i) which routes to operate, (ii) how often each route should run, (iii) the timetable of departures, and (iv) how vehicles are assigned. The problem is computationally hard, and real planners still rely heavily on manual judgement and spreadsheets.

The **Public Transit Planner** developed in this project is a decision-support pipeline that addresses these decisions using core ideas from *Data Structures and Algorithms*: graphs for the road network, a priority queue (binary heap) for Dijkstra's shortest-path algorithm, a greedy set-cover-style heuristic for route selection, and a marginal-benefit greedy rule for fleet allocation. The pipeline cleans raw data, builds the network, generates candidate routes, selects and schedules them within fleet and budget limits, and evaluates each plan using passenger-centred and operator-centred metrics.

Organisation of the report. Section 3 reviews related work; Section 4 states the problem and objectives; Section 5 lists hardware and software requirements; Section 6 presents the implementation modules; Section 7 reports and discusses results; Section 8 concludes and outlines future scope; Section 9 lists references.

3. LITERATURE SURVEY

The following eight related works and tools were studied. Their links are listed in the References section.

[1] Owais (2025) – *Transit network design problem: a half century of methodological research.*

A review of more than 170 studies from 1970 to 2024 on the transit route network design problem and the frequency setting problem. It maps how methods moved from classical optimisation models towards metaheuristics and, more recently, AI/ML, and highlights integrating route design with frequency setting as an open frontier. It motivated our decision to solve route selection and frequency allocation together under shared fleet and budget limits.

[2] Duran Micco & Vansteenwegen (2022) – *A survey on the transit network design and frequency setting problem.*

A survey in the journal *Public Transport* that compares assumptions, constraints, objectives, solution approaches and test instances used for the TNDFSP. It notes that heavy simplification means few optimisation techniques are usable in practice and that results are hard to compare across studies. We therefore state our assumptions explicitly and compare scenarios on identical data and metrics.

[3] Guihaire & Hao (2008) – *Transit network design and scheduling: a global review.*

A review that classifies 69 approaches covering route design, frequency setting and timetabling, and explains that coverage and directness affect service quality while network structure drives operating cost. It also frames the planning process as sequential steps (routes, frequencies, timetables, vehicle scheduling). Our pipeline follows this sequence and reports coverage, waiting, travel time and cost together.

[4] Delling, Pajor & Werneck (2012/2015) – *Round-Based Public Transit Routing (RAPTOR).*

Introduces RAPTOR, a round-based router that is not Dijkstra-based, scans each route at most once per round and finds Pareto-optimal journeys on arrival time and number of transfers. It shows that journey planning can be made fast without preprocessing. We use the same two criteria (time and transfers) when evaluating passenger journeys, but with a line-expanded Dijkstra search, which is simpler to implement and sufficient for a network of this size.

[5] General Transit Feed Specification (GTFS) – reference documentation.

GTFS is the open, widely adopted format in which agencies publish stops, routes, trips and stop times. Our stop, link and demand tables follow the same idea of normalised tabular transit data, so the planner can be extended to ingest real GTFS feeds in the future.

[6] OpenTripPlanner – open-source multimodal trip planner.

An open-source platform that builds a routable graph from street and GTFS data and answers passenger journey queries. It focuses on passenger-side trip planning rather than agency-side design under fleet and budget limits, which is the gap our planner targets.

[7] NetworkX – Python library for graph analysis.

A general graph library offering shortest paths, connectivity and centrality routines. We implemented Dijkstra's algorithm directly with a binary heap for the DSA course objective, but NetworkX provides a convenient reference for validating results on larger graphs.

[8] Dijkstra (1959) – *A note on two problems in connexion with graphs.*

The classic paper giving the shortest-path algorithm that underlies most routing engines. With a binary heap it runs in $O((V+E) \log V)$. It is the core routine used in our candidate-route generation, in the passenger journey evaluation and in the connectivity checks.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Design bus routes, frequencies and timetables that maximise service coverage and minimise passenger travel time and operating cost within a fixed fleet size and daily budget.

Problem Description: City planners must decide which stops to connect, how many buses each route gets and when buses depart, but demand is uneven, travel times vary and resources are limited. Manual planning cannot evaluate the huge number of possible route and frequency combinations.

Objectives:

The project was initiated to give planners a repeatable, data-driven way to compare transit plans instead of relying on intuition. The objectives are:

- To collect and clean stop, travel-time and passenger-demand data so that it is accurate and consistent.
- To model the city as a weighted graph and use Dijkstra's algorithm to generate candidate routes along demand corridors.
- To select a set of routes and allocate buses such that the fleet size and daily budget are never exceeded, while maximising covered demand.
- To generate practical timetables (departure times by period) from demand patterns and vehicle availability.
- To evaluate plans using route/stop coverage, average waiting time, travel time, fleet utilisation and operating expense.
- To compare alternative planning scenarios through tables and visualisations that support decision making.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

Component	Minimum	Recommended
Processor	Dual-core, 2.0 GHz	Quad-core (e.g. Intel i5 / Ryzen 5) or better
RAM	4 GB	8 GB
Storage	500 MB free	1 GB free
Display	1366 x 768	1920 x 1080 (for maps and charts)

Software Requirements:

Item	Details
Operating system	Windows 10/11, Ubuntu 22.04+ or macOS
Language	Python 3.12
Libraries	NumPy 2.4, pandas 3.0, Matplotlib 3.10 (standard library: heapq, math, json)
Report generation	ReportLab 4.4
Editor / IDE	VS Code, PyCharm or Jupyter Notebook
Data format	CSV files (stops, links, demand) & GTFS-style tabular data

6. IMPLEMENTATION

The planner is implemented in Python as a pipeline of independent modules. A global `config.py` holds the assumptions: average speed 18 km/h, 0.7 min dwell per stop, 5 min terminal layover, 3 min penalty per transfer, cost Rs 60 per bus-km and Rs 350 per bus-hour, maximum headways of 30 min (peak) and 40 min (off-peak) and a minimum headway of 4 min. The four principal modules are shown below; the passenger-journey evaluator (`evaluate.py`) and the driver (`main.py`) tie them together.

Data used. No operator data was supplied, so a *synthetic city* of 36 stops was generated (gravity-model demand of about 48,000 trips/day, road links with congested travel times). Data-quality faults (duplicates, missing and negative values, outliers) were injected deliberately so that Module 1 could be tested. The same code accepts real CSV files with the same columns.

Module 1: Data collection and cleaning (`preprocess.py`)

```
"""Module 1 - Data collection & cleaning."""
import numpy as np, pandas as pd
from config import *

def clean(raw="data/raw", out="data/clean"):
    rep = {}
    stops = pd.read_csv(f"{raw}/stops.csv"); links = pd.read_csv(f"{raw}/links.csv")
    dem = pd.read_csv(f"{raw}/demand.csv")

    # --- stops: remove duplicates, impute missing coordinates from neighbours ---
    n0 = len(stops); stops = stops.drop_duplicates("stop_id").reset_index(drop=True)
    rep["duplicate_stops_removed"] = n0 - len(stops)
    miss = stops[stops.x.isna() | stops.y.isna()].stop_id.tolist()
    rep["missing_coordinates_imputed"] = len(miss)
    for s in miss:
        nb = links[(links.from_id == s) | (links.to_id == s)]
        ids = set(nb.from_id) | set(nb.to_id); ids.discard(s)
        ref = stops[stops.stop_id.isin(ids) & stops.x.notna()]
        stops.loc[stops.stop_id == s, ["x", "y"]] = ref[["x", "y"]].mean().values
    xy = stops.set_index("stop_id")[["x", "y"]]

    # --- links: undirected de-duplication, fix invalid / missing / outlier times ---
    links["key"] = links.apply(lambda r: tuple(sorted((r.from_id, r.to_id))), axis=1)
    n0 = len(links); links = links.drop_duplicates("key").reset_index(drop=True)
    rep["duplicate_links_removed"] = n0 - len(links)
    dist = lambda r: float(np.hypot(*(xy.loc[r.from_id] - xy.loc[r.to_id]).values))
    links["dist_km"] = links.apply(dist, axis=1)
    est = links.dist_km / SPEED_KMPH * 60 * 1.25 + DWELL_MIN # speed-based estimate
    bad_missing = links.time_min.isna(); bad_neg = links.time_min <= 0
    bad_out = links.time_min > 3 * est # > 3x plausible time
    rep.update(missing_times_imputed=int(bad_missing.sum()), invalid_times_fixed=int(bad_neg.sum()),
              outlier_times_fixed=int(bad_out.sum()))
    fix = bad_missing | bad_neg | bad_out
    links.loc[fix, "time_min"] = est[fix].round(2)
    links = links.drop(columns="key")

    # --- demand: drop self-pairs, repair negatives / missing ---
    n0 = len(dem); dem = dem[dem.origin != dem.dest].copy()
    rep["self_pair_rows_removed"] = n0 - len(dem)
    rep["negative_demand_zeroed"] = int((dem.trips < 0).sum())
    dem.loc[dem.trips < 0, "trips"] = 0
    rep["missing_demand_imputed"] = int(dem.trips.isna().sum())
    med = dem.groupby("origin").trips.transform(lambda s: s[s > 0].median())
    dem["trips"] = dem.trips.fillna(med).round().astype(int)

    stops.to_csv(f"{out}/stops.csv", index=False); links.to_csv(f"{out}/links.csv", index=False)
    dem.to_csv(f"{out}/demand.csv", index=False)
    return stops, links, dem, rep
```

Duplicates are removed, missing coordinates are inferred from neighbouring stops, invalid or outlier link times are replaced by a speed-based estimate, and repaired demand values are imputed, so every later module works on consistent data.

Module 2: Graph construction and Dijkstra routing (`graph_routing.py`)

```

"""Module 2 - Graph construction & shortest-path routing (Dijkstra)."""
import heapq, numpy as np

def build_graph(stops, links):
    ids = stops.stop_id.tolist(); adj = {s: [] for s in ids}
    for r in links.itertuples():
        adj[r.from_id].append((r.to_id, r.time_min, r.dist_km))
        adj[r.to_id].append((r.from_id, r.time_min, r.dist_km))
    return ids, adj

def dijkstra(adj, src):
    """Single-source shortest travel time with a binary heap,  $O((V+E) \log V)$ ."""
    dist = {v: float("inf") for v in adj}; prev = {}; dist[src] = 0.0
    pq = [(0.0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        for v, t, _ in adj[u]:
            if d + t < dist[v]:
                dist[v] = d + t; prev[v] = u; heapq.heappush(pq, (dist[v], v))
    return dist, prev

def path_to(prev, src, dst):
    p = [dst]
    while p[-1] != src: p.append(prev[p[-1]])
    return p[::-1]

def link_lookup(adj):
    return {(u, v): (t, k) for u in adj for v, t, k in adj[u]}

```

The road network is an adjacency list. Dijkstra's algorithm with a binary heap gives all shortest travel times and paths in $O((V+E) \log V)$ per source.

Module 3: Route selection and fleet allocation (optimizer.py)

```

"""Module 3 - Route generation, selection and fleet allocation under fleet & budget limits."""
import math, numpy as np
from config import *
from graph_routing import dijkstra, path_to
from timetable import route_stats, daily_cost

def candidate_routes(ids, adj, demand, hubs, top_k=70):
    sp = {s: dijkstra(adj, s) for s in ids}
    cands = {}
    def add(p, minlen):
        if len(p) >= minlen:
            cands[tuple(p) if p[0] < p[-1] else tuple(p[::-1])] = p
    pairs = sorted([(v, o, d) for (o, d), v in demand.items() if o < d], reverse=True)
    for v, o, d in pairs[:top_k * 4]:
        add(path_to(sp[o][1], o, d), 6)
        if len(cands) >= top_k: break
    for s in ids:
        # feeder candidates so every stop can be covered
        for h in hubs:
            if s != h: add(path_to(sp[h][1], h, s), 3)
    return list(cands.values()), sp

def greedy_select(cands, adj, demand, ids, fleet, budget, w_stop):
    """Pick routes maximising (new direct demand + stop-coverage value) per bus needed."""
    chosen, covered, served = [], set(), set()
    used, cost = 0, 0.0
    while True:
        best = None
        for p in cands:
            if p in chosen: continue
            st = route_stats(p, adj); need = math.ceil(st["rt"] / MAX_HEADWAY_PEAK)
            c = daily_cost(st, need)
            if used + need > fleet or cost + c > budget: continue
            S = set(p)
            gain = sum(v for (o, d), v in demand.items() if o < d and o in S and d in S
                       and (o, d) not in served)
            gain += sum(w_stop[s] for s in S - covered)
            score = gain / need
            if best is None or score > best[0]: best = (score, p, need, c, gain)
            if best is None or best[4] <= 0: break
        _, p, need, c, _ = best; chosen.append(p); used += need; cost += c
        S = set(p); covered |= S
        served |= {(o, d) for (o, d) in demand if o < d and o in S and d in S}
    return chosen

def allocate_buses(routes, adj, demand, fleet, budget):
    """Distribute spare buses where passenger-minutes of waiting saved are largest."""
    st = [route_stats(p, adj) for p in routes]

```

```

load = np.zeros(len(routes))
for (o, d), v in demand.items():
    if o < d:
        srv = [i for i, p in enumerate(routes) if o in p and d in p]
        for i in srv: load[i] += 2 * v / len(srv) # both directions
b = [math.ceil(st["rt"] / MAX_HEADWAY_PEAK) for s in st]
cost = sum(daily_cost(s, k) for s, k in zip(st, b))
while sum(b) < fleet:
    best, bi = 0.0, None
    for i, s in enumerate(st):
        if s["rt"] / (b[i] + 1) < MIN_HEADWAY: continue
        dc = daily_cost(s, b[i] + 1) - daily_cost(s, b[i])
        if cost + dc > budget: continue
        saved = load[i] * (s["rt"] / b[i] - s["rt"] / (b[i] + 1)) / 2
        if saved / dc > best: best, bi = saved / dc, i
    if bi is None: break
    cost += daily_cost(st[bi], b[bi] + 1) - daily_cost(st[bi], b[bi]); b[bi] += 1
return b, load

```

Candidate routes are shortest paths between the highest-demand stop pairs plus feeder paths from every stop to the busiest hubs. A greedy rule then picks the route with the best (new direct demand + stop-coverage value) per bus required, until fleet or budget would be exceeded. Spare buses are given, one at a time, to the route where waiting-time saved per rupee spent is largest, which makes buses grow roughly with the square root of route demand.

Module 4: Timetable generation and cost model (timetable.py)

```

"""Module 4 - Timetable generation and cost model."""
import math
from config import *

def route_stats(path, adj):
    lk = {}
    for u in path[:-1]:
        for v, t, k in adj[u]: lk[(u, v)] = (t, k)
    ride = sum(lk[(a, b)][0] for a, b in zip(path, path[1:]))
    km = sum(lk[(a, b)][1] for a, b in zip(path, path[1:]))
    return dict(ride=ride, km=km, rt=2 * ride + 2 * LAYOVER_MIN) # round-trip time

def period_buses(st, b_peak, mult):
    return max(math.ceil(st["rt"] / MAX_HEADWAY_OFF), math.ceil(b_peak * mult)) if mult < 1 \
        else max(b_peak, math.ceil(st["rt"] / MAX_HEADWAY_PEAK))

def timetable(st, b_peak):
    """Departure times (minutes after midnight) for both terminals, per period."""
    out = {}
    for name, t0, t1, mult in PERIODS:
        b = period_buses(st, b_peak, mult); h = st["rt"] / b
        dep = []; t = t0 * 60
        while t < t1 * 60 - 1e-9:
            dep.append((round(t), round(t + h / 2))); t += h # (A->B, B->A)
        out[name] = dict(buses=b, headway=h, departures=dep)
    return out

def daily_cost(st, b_peak):
    tt = timetable(st, b_peak); trips = bus_hours = 0
    for (name, t0, t1, _), v in zip(PERIODS, tt.values()):
        trips += 2 * len(v["departures"]); bus_hours += v["buses"] * (t1 - t0)
    return trips * st["km"] * COST_PER_KM + bus_hours * COST_PER_HOUR

```

For each of four service periods (AM peak, midday, PM peak, evening) the number of buses sets the headway (round-trip time divided by buses). Departure times are generated for both terminals and the daily cost is bus-km x Rs 60 plus bus-hours x Rs 350.

7. RESULTS & DISCUSSION

7.1 Data cleaning

Running Module 1 on the raw files produced the following repairs. After cleaning, the network is connected (checked by running Dijkstra from one stop to all others) and all 1,260 origin-destination pairs have valid demand values.

Data-quality problem	Records fixed
Duplicate stops removed	2
Missing stop coordinates imputed	2
Duplicate road links removed	4
Missing link travel times imputed	3
Zero/negative link times corrected	3
Outlier link times corrected	3
Self-pair demand rows removed	1
Negative demand values zeroed	3
Missing demand values imputed	5

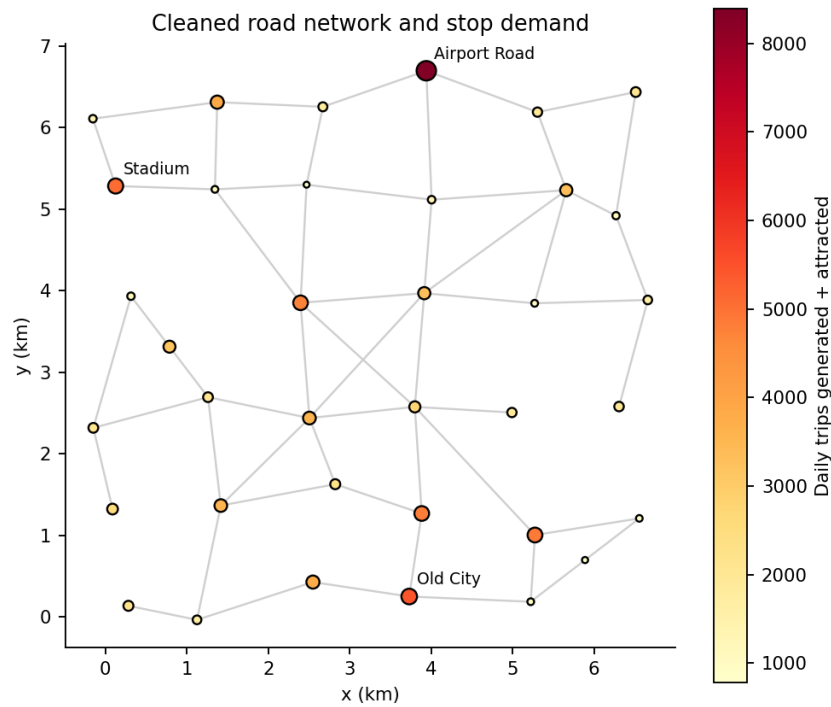


Figure 1: Cleaned road network. Marker size and colour show daily trips generated and attracted at each stop; hubs are labelled.

7.2 Route plans

Scenario A is a manual radial plan with spokes from the busiest hub and equal buses per spoke (fleet 48). Scenario B is the planner's output for the same fleet and the same daily budget (Rs 519,343). Scenario C allows a fleet of 66 and a 40% larger budget; Scenario D keeps the fleet at 48 but cuts the budget by 25%.

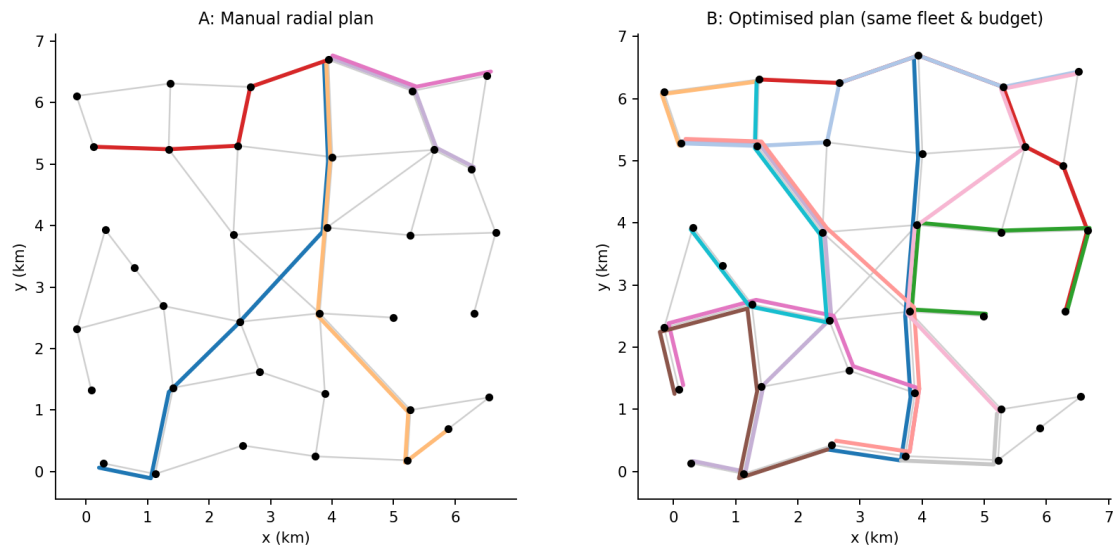


Figure 2: Manual radial plan (left) versus the optimised plan (right). The optimised routes reach almost every stop; the radial plan leaves most of the city unserved.

7.3 Scenario comparison

Metric	A Manual	B Optimised	C Fleet 66	D -25% budget
Routes operated	5	12	18	10
Buses used (fleet limit)	45	34	48	27
Stop coverage (%)	52.8	94.4	100.0	97.2
Demand served (%)	27.7	96.2	100.0	97.9
Direct trips, no transfer (%)	16.3	49.3	57.7	40.7
Served within 1 transfer (%)	27.7	90.2	95.5	82.9
Avg. waiting time (min)	5.2	20.0	18.6	21.7
Avg. in-vehicle time (min)	22.0	22.1	20.5	24.6
Avg. total travel time (min)	28.4	43.8	40.6	48.6
Avg. transfers per trip	0.41	0.55	0.47	0.75
Peak fleet utilisation (%)	93.8	70.8	72.7	56.2
Daily fleet utilisation (%)	67.4	63.4	66.4	48.8
Daily operating cost (Rs)	519,343	514,454	725,743	387,608
Passenger trips served / day	13,288	46,160	47,964	46,978
Cost per passenger trip (Rs)	39.1	11.1	15.1	8.3

Table 1: Performance of the four planning scenarios. Averages are over passengers whose trips are served.

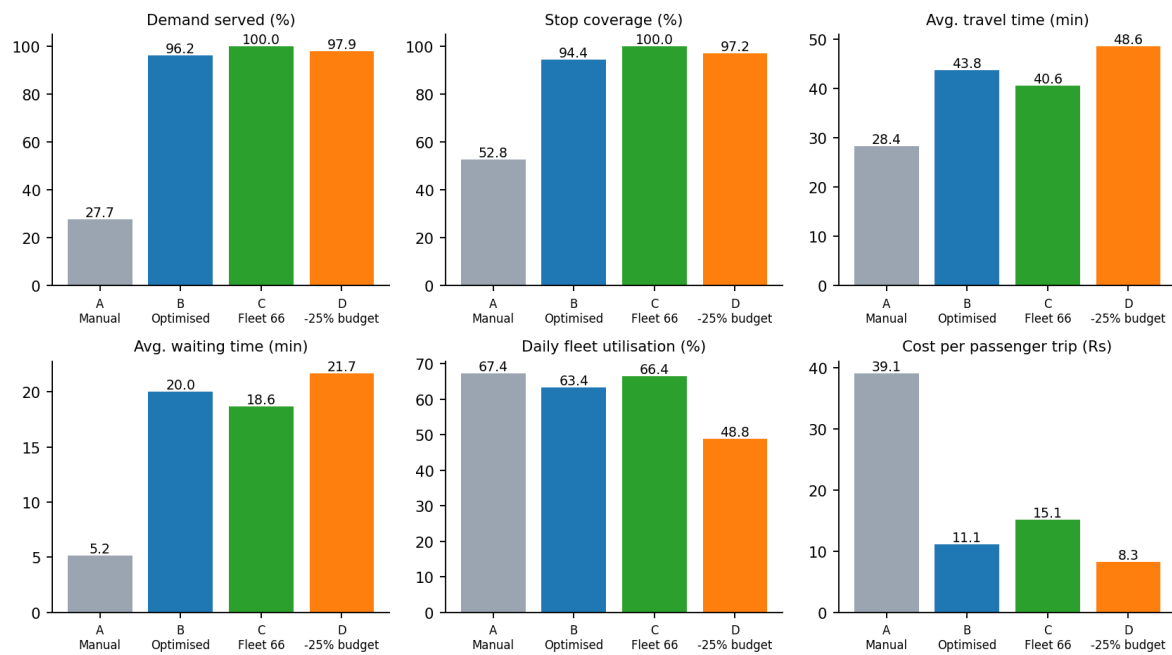


Figure 3: Scenario comparison across six key metrics.

Observations. With the same fleet size and almost the same daily budget, the optimised plan B serves **96.2%** of passenger demand compared with **27.7%** for the manual plan, and covers 94% of stops instead of 53%. Cost per passenger trip falls from Rs 39.1 to Rs 11.1 (a 71% reduction). Scenario C, with more buses and money, reaches full stop and demand coverage and lowers average travel time to 40.6 min. Scenario D shows the trade-off of a tight budget: it still serves 98% of demand at the lowest cost per trip, but with fewer buses in use (27 of 48), longer travel times and more transfers.

Important caveat on waiting and travel time. The manual plan shows a lower average waiting time (5.2 min) and a lower average travel time (28.4 min) than the optimised plan (20.0 and 43.8 min). This is not a better service: these averages are taken only over the 28% of passengers the manual plan can serve at all (mostly short trips near the hub). The optimised plan carries 3.5 times as many passengers, including long cross-city trips and trips needing a transfer, so its averages are naturally higher. Several optimised routes also run at headways close to the 30-minute maximum (Table 2), because the budget is spent first on coverage. A fairer comparison for future work is a weighted metric that penalises unserved demand.

7.4 Routes, fleet and timetable of the optimised plan (Scenario B)

Route	From	To	Stops	Round trip (min)	Buses	Peak headway	Pass./day
R1	Airport Road	Stop 03	7	84	3	28	3,114
R2	Stadium	Stop 32	3	30	1	30	852
R3	Stop 18	Stop 32	8	87	4	22	3,462
R4	Stadium	Stop 01	7	90	3	30	2,077
R5	Old City	Stop 07	7	74	3	25	2,335
R6	Old City	General Hospital	3	34	2	17	800
R7	Stop 19	Stop 32	7	81	3	27	2,136
R8	Stadium	Stop 36	7	76	3	25	2,299

Route	From	To	Stops	Round trip (min)	Buses	Peak headway	Pass./day
R9	Stop 17	Stop 18	6	76	3	25	865
R10	Stadium	Stop 03	7	89	3	30	2,494
R11	Stop 03	Stop 07	6	72	3	24	1,683
R12	General Hospital	Stop 36	6	80	3	27	1,515

Table 2: Routes of the optimised plan with allocated buses; peak headway is in minutes and Pass./day is daily passengers on the route.

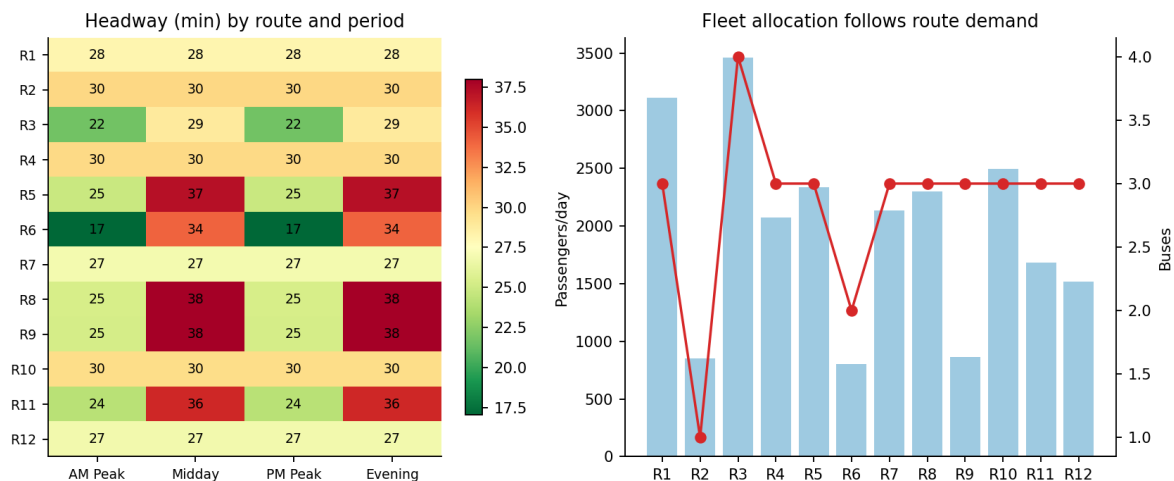


Figure 4: Headway by route and period (left) and buses allocated against route demand (right). Busy routes receive more buses and shorter headways; headways lengthen in midday and evening.

Sample timetable for the busiest route, R3 (Stop 18 ↔ Stop 32, 8 stops):

Period	Buses	Headway (min)	First departures from Stop 18
AM Peak	4	21.6	06:00, 06:22, 06:43, 07:05, 07:27, 07:48 ...
Midday	3	28.9	09:00, 09:29, 09:58, 10:27, 10:55, 11:24 ...
PM Peak	4	21.6	16:00, 16:22, 16:43, 17:05, 17:27, 17:48 ...
Evening	3	28.9	19:30, 19:59, 20:28, 20:57, 21:25, 21:54 ...

Table 3: Generated timetable (24-hour clock).

7.5 Program output

```
cleaning: {'duplicate_stops_removed': 2, 'missing_coordinates_imputed': 2, 'duplicate_links_removed': 4, 'missing_times_imputed': 0}
```

```
A: routes= 5 buses=45/48 demand_served=27.7% wait=5.2min travel=28.4min cost=Rs 519,343
B: routes=12 buses=34/48 demand_served=96.2% wait=20.0min travel=43.8min cost=Rs 514,454
C: routes=18 buses=48/66 demand_served=100.0% wait=18.6min travel=40.6min cost=Rs 725,743
D: routes=10 buses=27/48 demand_served=97.9% wait=21.7min travel=48.6min cost=Rs 387,608
```

The console output of `main.py` confirms the cleaning counts and the headline metrics of the four scenarios reported in Table 1.

7.6 Discussion and limitations

The results show that a demand-aware, budget-aware planner can raise coverage and reduce cost per passenger far beyond a simple radial layout, using the same fleet and money. The following limitations should be kept in mind: (1) the data is synthetic, so absolute numbers are illustrative and the study demonstrates the method, not the performance of a real city; (2) route selection and bus allocation are greedy heuristics, so they are fast and explainable but not guaranteed to be globally optimal; (3)

demand is assumed fixed and does not respond to service quality; (4) loads are estimated by splitting demand equally among routes that serve a pair, and vehicle scheduling uses a simple average round-trip time without traffic variation.

8. CONCLUSION & FUTURE SCOPE

The project set out to build a data-driven planner that designs routes and timetables within fixed fleet and budget limits. The objectives were met: the data pipeline repaired all injected faults; Dijkstra-based routing generated candidate corridors; the greedy selector and allocator produced plans that never exceeded the fleet or budget; timetables were generated for four service periods; and the evaluator reported coverage, waiting time, travel time, fleet utilisation and cost. At equal fleet and budget the optimised plan served 96% of demand versus 28% for the manual plan and cut cost per passenger trip by 71%, and the scenario comparison made the fleet-versus-budget trade-offs explicit.

Future scope

Near term

- Import real GTFS feeds and survey/smart-card demand data in place of synthetic data.
- Add an interactive dashboard (map, sliders for fleet and budget) for planners.
- Add a weighted objective that penalises unserved passengers when comparing scenarios.

Medium term

- Replace the greedy search with metaheuristics (genetic algorithm, simulated annealing) or integer programming and compare solution quality.
- Model time-dependent travel times, vehicle blocking and driver shifts, and demand elasticity.
- Use RAPTOR-style routing for faster multi-criteria passenger journey evaluation on large networks.

Long term

- Predict demand with machine learning and re-plan routes and timetables continuously from live data.
- Extend to multimodal planning (metro, feeder buses, bike-share) with equity and emissions objectives.

9. REFERENCES

1. M. Owais, “Transit network design problem: a half century of methodological research,” *Innovative Infrastructure Solutions*. <https://aun.edu.eg/engineering/node/39797>
2. J. Duran Micco and P. Vansteenwegen, “A survey on the transit network design and frequency setting problem,” *Public Transport*, vol. 14, pp. 155–190, 2022. <https://lirias.kuleuven.be/3622915>
3. V. Guihaire and J.-K. Hao, “Transit network design and scheduling: A global review,” *Transportation Research Part A*, 2008. <https://www.worldtransitresearch.info/research/1874>
4. D. Delling, T. Pajor and R. F. Werneck, “Round-Based Public Transit Routing,” *Transportation Science*, 2015 (ALENEX 2012). <https://doi.org/10.1287/trsc.2014.0534>
5. General Transit Feed Specification (GTFS) Schedule Reference. <https://gtfs.org/documentation/schedule/reference/>
6. OpenTripPlanner – open-source multimodal trip planner. <https://www.opentripplanner.org/>
7. NetworkX – Network Analysis in Python (documentation). <https://networkx.org/documentation/stable/>
8. E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. <https://doi.org/10.1007/BF01386390>
9. pandas – Python Data Analysis Library (documentation). <https://pandas.pydata.org/docs/>
10. Matplotlib – Visualization with Python (documentation). <https://matplotlib.org/stable/>